UANL
UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FIME
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Nombres Josue Yahir Vela Hernandez Osweld Omar Lozano Martínez Sylvia Yadira González Parra	**Matriculas** 2142881 2142366 2142962
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor:** Ing. Miguel Ángel Gómez Marroquin
Actividad: Diseño Arquitectural del sistema	**Fecha:** 17/10/2026

Contenido

1. Alcance ***3***

2. Casos de uso ***3***

3. Comparación de patrones ***4***

4. Mapa de Dominio ***6***

4.1 Diagrama 6

4.2 Glosario Actualizado 7

4.3 Fichas de Aggregate 7

4.4 Value Objects, Domain Services, Domain Events 8

4.5 Explicacion de decisiones dudosas o descartadas 9

5. Service Layer ***10***

5.1 Revisión de service Layer 11

6. Repository ***12***

7. Unit of Work ***13***

8. CQRS ***13***

9. Revisión SOLID ***14***

10. Representación de la arquitectura ***15***

10.1 Diagrama de clases 15

10.2 Diagramas de secuencia 16

10.3 Architecture Decision Records 17

11. Validación de la propuesta ***18***

12. Matriz de trazabilidad ***19***

13. Bitácora de IA ***21***

14. Conclusiones ***25***

1. Alcance

Este informe toma como base el BRD, las user stories, los criterios de aceptación, el BPMN y el modelo de dominio entregados en la Unidad 2 del caso AutoFix Pro, un taller mecánico que atiende clientes particulares y de flotilla. Antes de diseñar, se revisó ese material y se detectaron los siguientes hallazgos:

1) El Gerente del taller aparece como stakeholder y en US-15, pero no tiene carril propio en el BPMN.
2) La pregunta 14 del levantamiento deja explícitamente abierta la política para vehículos no recogidos a tiempo; se declara como supuesto pendiente de validar.
3) Las restricciones de tablets/iPads con conexión inestable no tenían, en la Unidad 2, ninguna decisión de persistencia asociada; se retoma en la Etapa 5.
4) US-10 combina, en un solo enunciado, la regla de "falla adicional" (pregunta 10 del levantamiento) con el umbral del 20% de "ampliación de trabajos" (pregunta 12); se declara la ambigüedad y se opta por aplicar el umbral solo a ampliaciones sobre trabajos ya autorizados.

Se verificaron también dos archivos fuente adicionales (el .drawio del BPMN y el UML) contra el PDF final. El BPMN fuente confirmó que el diagrama del PDF es consistente con las user stories.

2. Casos de uso

A continuación se presenta la tabla con los cinco casos de uso seleccionados (`CU-01` a `CU-05`) para el taller mecánico AutoFix Pro, los cuales abarcan desde la cotización inicial y fallas adicionales hasta la gestión de refacciones sin existencia, cierre de caja y facturación consolidada de flotillas, vinculando sus actores, reglas de negocio y resultados esperados.

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Generar Cotización Inicial y Autorización Digital (total o parcial)	Asesor de servicio / Cliente	No se cotiza sin diagnóstico previo; solo trabajos autorizados pasan a ejecución (US-08)	La orden pasa a Cotizado; trabajos autorizados quedan con evidencia digital
CU-02	Falla Adicional y Cotización Complementaria	Técnico / Asesor / Cliente	Toda falla adicional se reporta antes de continuar; el trabajo no se ejecuta sin autorización (US-09/10)	Cotización complementaria generada; trabajo en Pendiente de autorización
CU-03	Refacción sin Existencia y En Espera de Refacción	Compras / Asesor de servicio	Compras genera orden de compra si falta refacción; cliente notificado de nueva fecha (US-13)	Vehículo marcado En espera de refacción; cliente notificado

CU-04	Cierre de Caja	Caja	Solo se cobran conceptos autorizados y ejecutados (US-19)	Orden cambia a Cerrado solo con lo autorizado y ejecutado
CU-05	Autorización Delegada y Facturación Consolidada de Flotilla	Responsable de cuenta / Caja	Solo el responsable de cuenta autoriza en nombre de la flotilla; facturación agrupa órdenes cerradas del mes (US-17)	Autorización con rol Responsable de flotilla; documento de cobro consolidado

3. Comparación de patrones

En la siguiente tabla se comparan los patrones *Transaction Script*, *Table Module* y *Domain Model* frente a la complejidad de AutoFix Pro, evaluando el manejo de los 8 estados de la orden de servicio, la diferenciación entre cliente particular y flotilla (`ClienteFlotilla`) y el umbral del 20% con base en la evidencia de la Unidad 2.

Criterio	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Reglas repetidas en cada script, sin lugar central	Sin mecanismo para reglas transversales	Encapsula cada regla en el objeto correspondiente	BN-06; criterio US-03; pregunta 12
Cambios esperados	Cada cambio obliga a reescribir scripts	No facilita absorber reglas nuevas de estado	Comportamiento nuevo sin tocar el resto	Pregunta 14 (brecha); restricción de conexión inestable
Interacción entre conceptos	No modela relaciones, solo pasos	Trabaja sobre filas, no relaciones	Relaciones ya definidas en el UML de U2	Sección 6.1 UML (Genera, Requiere, Contiene...)
Estados e invariantes	Sin máquina de estados central	No maneja invariantes	Entidades protegen su propia máquina de estados	US-14 (8 estados); US-07; US-08

Diferenciación por tipo de cliente	Condicionales repetidos por tipo	Filtra sin encapsular comportamiento	ClienteFlotilla como especialización	BN-05; ClienteFlotilla en UML U2; US-16,17,18

Decision: Domain Model como patrón principal del flujo de servicio (OrdenDeServicio, Cotización, Trabajo,Autorización, Cliente/ClienteFlotilla). Como excepción delimitada, el módulo de reportes (US-15) es candidato a un Table Module ligero, porque ahí no hay reglas de negocio que proteger, solo consulta sobre datos ya consolidados.

AutoFix Pro no es un caso de pocas reglas estables. El levantamiento documenta al menos cuatro mecanismos que cambian el comportamiento del sistema según el contexto: el umbral del 20% para ampliaciones (pregunta 12), las 24 horas de plazo antes de escalar (pregunta 13), la autorización parcial por trabajo individual (US-05/06) y el tratamiento diferenciado de clientes de flotilla frente a particulares (BN-05). Un Transaction Script resolvería cada caso de uso como una secuencia de pasos independiente; eso obligaría a repetir la verificación "¿este cliente es de flotilla?" y "¿esta ampliación supera el 20%?" en cada script que toque una autorización, con el riesgo real de que una regla se actualice en un lugar y se olvide en otro. El costo que Domain Model introduce, en cambio, es desarrollo inicial más lento: hay que diseñar Aggregates, invariantes y Value Objects antes de poder ejecutar el primer caso de uso completo, y el equipo necesita entender el patrón para no convertir las entidades en simples contenedores de datos.

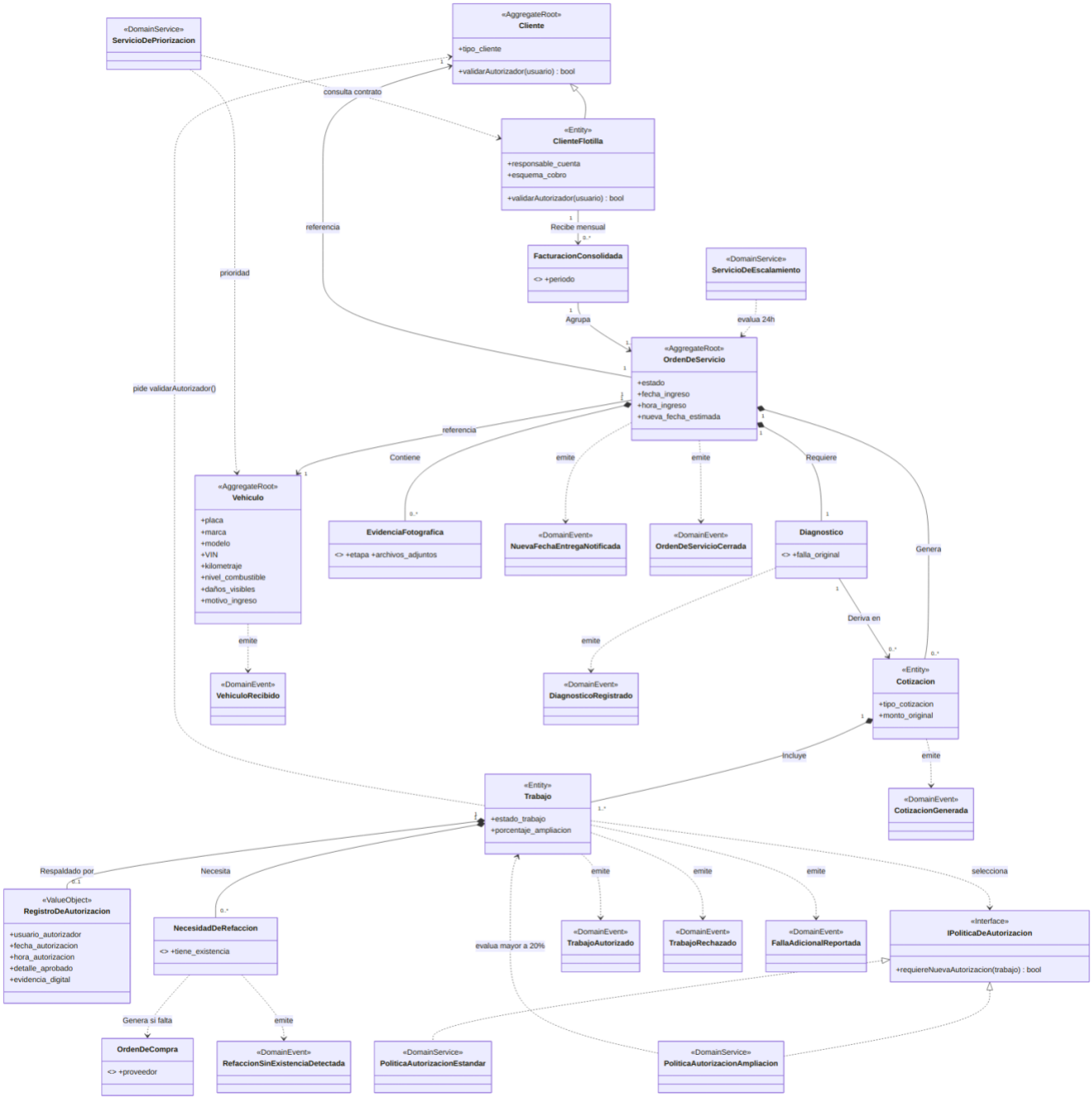
Table Module se descartó como patrón principal porque el caso no gira en torno a operar sobre conjuntos de registros de una tabla, sino en proteger una secuencia de estados por instancia: una OrdenDeServicio individual pasa por 9 estados (Recibido, Diagnosticado, Cotizado, En espera de autorización, En reparación, En espera de refacción, En espera de cliente, Listo para entrega, Cerrado), y la regla "no se cierra con trabajos sin resolver" depende del historial de esa orden en particular, no de una condición que se pueda expresar como un filtro sobre una tabla completa. Donde sí es razonable Table Module es en US-15 (reportes e indicadores): ahí la necesidad real es agregar datos ya existentes (número de disputas, tiempo promedio de comunicación, porcentaje de evidencia digital) sin proteger ninguna regla de negocio nueva, así que forzar ese módulo a pasar por el Aggregate completo de OrdenDeServicio solo agregaría complejidad sin beneficio.

El costo concreto de elegir Domain Model como patrón principal es que la Unidad 3 debe invertir tiempo en decidir dónde vive cada responsabilidad (ver Etapas 3 a 7) antes de escribir una sola línea de infraestructura; ese costo se paga una sola vez, mientras que el costo de un Transaction Script —reglas duplicadas y posibilidad de inconsistencia entre pantallas— se repetiría cada vez que el negocio agregue una excepción nueva, algo que el propio levantamiento ya anticipa (pregunta 14, política de vehículo no recogido, aún sin resolver).

4. Mapa de Dominio

A continuación, se presenta el mapa de dominio de **AutoFix Pro**, donde se estructuran sus cinco *Aggregate Roots* (`OrdenDeServicio`, `Vehiculo`, `Cliente/ClienteFlotilla`, `OrdenDeCompra` y `FacturacionConsolidada`), junto con los *Value Objects*, *Domain Services* (`IPoliticaDeAutorizacion`) y *Domain Events* que protegen las invariantes del taller.

4.1 Diagrama



4.2 Glosario Actualizado

Termino	Descripción
Aggregate	Grupo de objetos del dominio con un límite de consistencia; solo se modifica a través de su raíz
Raíz de Aggregate (Aggregate Root)	Objeto único que controla el acceso y los cambios dentro de un Aggregate; los demás objetos del grupo no se modifican directamente desde fuera
Value Object	Objeto sin identidad propia, definido por sus valores, inmutable una vez creado
Domain Service	Comportamiento del dominio que no pertenece naturalmente a una sola entidad
Domain Event	Hecho de negocio ya ocurrido, comunicado a otros componentes del sistema
Registro de Autorización (corregido)	Realización concreta, en el modelo de dominio, del término “Autorización Digital” del glosario de la U2: Value Object inmutable adjunto a cada Trabajo
Necesidad de Refacción (corregido)	Realización, en el modelo de dominio, del término “Refacción sin Existencia”: Value Object adjunto a un Trabajo, no una entidad de inventario completo

4.3 Fichas de Aggregate

Aggregate	Responsabilidades	Invariantes	Eventos posibles
OrdenDeServicio (raíz)	coordinar el ciclo de vida completo del servicio, desde el ingreso hasta el cierre.	no puede generarse una Cotización sin un Diagnóstico previo; un Trabajo no se ejecuta si su estado no es “Autorizado”; la orden no cambia a “Cerrado” si	`DiagnosticoRegistrado`, `CotizacionGenerada`, `TrabajoAutorizado`, `TrabajoRechazado`, `FallaAdicionalReportada`, `NuevaFechaEntregaNotificada`, `OrdenDeServicioCerrada`.

		quedan Trabajos sin resolver.	
Vehículo (raíz)	representar al vehículo de forma independiente de cualquier orden de servicio puntual, permitiendo consultar su historial.	la placa y el VIN no cambian una vez registrados.	`VehiculoRecibido`.
Cliente / ClienteFlotilla (raíz)	distinguir reglas de autorización y facturación entre clientes particulares y de flotilla.	solo el responsable_cuenta registrado puede autorizar en nombre de una flotilla.	ninguno propio; participa como referencia en eventos de otros Aggregates.
OrdenDeCompra (raíz)	gestionar la solicitud de una refacción faltante ante un proveedor, de forma independiente al ritmo de la orden de servicio que la originó.	solo se crea cuando una NecesidadDeRefaccion indica tiene_existencia = falso.	`RefaccionSinExistenciaDetectada`.
FacturacionConsolidada (raíz)	agrupar, por periodo, las órdenes de servicio cerradas de una cuenta de flotilla en un solo documento de cobro.	solo agrupa órdenes en estado “Cerrado”; no incluye dos veces la misma orden en un mismo periodo.	`OrdenDeServicioCerrada`.

4.4 Value Objects, Domain Services, Domain Events

- **Value Objects:** `RegistroDeAutorizacion` (inmutable, US-07), `EvidenciaFotografica` (inmutable), `NecesidadDeRefaccion`.
- **Domain Services:** `IPoliticaDeAutorizacion` con dos implementaciones (estándar y por ampliación >20%), `ServicioDeEscalamiento` (24h), `ServicioDePriorizacion` (prioridad contractual de flotilla).
- **Domain Events:** `VehiculoRecibido`, `DiagnosticoRegistrado`, `CotizacionGenerada`, `TrabajoAutorizado`, `TrabajoRechazado`, `FallaAdicionalReportada`, `RefaccionSinExistenciaDetectada`, `NuevaFechaEntregaNotificada`, `OrdenDeServicioCerrada`.

4.5 Explicacion de decisiones dudosas o descartadas

1. **Vehículo como Aggregate separado de OrdenDeServicio** (y no contenido en ella): un vehículo puede tener varias órdenes a lo largo del tiempo; mantenerlos separados evita un Aggregate gigante y obliga a referenciarlo por id, no por contención.

2. **Autorización modelada como Value Object (`RegistroDeAutorizacion`), no como entidad propia:** no necesita identidad distinta de sus valores, y puede haber varias por Trabajo a lo largo del tiempo (autorizaciones parciales), lo cual encaja con la definición de Value Object inmutable.

3. **`prioridad_contractual` removida como atributo almacenado de Vehículo** (presente en el UML original de la U2) y sustituida por el Domain Service `ServicioDePriorizacion`, que la calcula a partir del contrato de `ClienteFlotilla`. Guardarla en Vehículo duplicaría información que puede cambiar en el contrato y quedar desactualizada.

4. **`falla_adicional` removida del atributo de Diagnóstico** (donde la U2 la había colocado junto a `falla_original`) y modelada en su lugar como el evento `FallaAdicionalReportada`, que origina un nuevo Trabajo y una Cotización Complementaria. Esto permite varias fallas adicionales durante una misma reparación, cosa que un solo campo de texto no soportaba.

5. **Refacción modelada como Value Object (`NecesidadDeRefaccion`) y no como entidad de inventario completo**, porque la gestión de stock, ubicaciones y catálogo de proveedores queda fuera del alcance declarado en el BRD (sección “No incluye”).

6. **Pendiente de validar con el stakeholder (no resuelto, se declara como supuesto):** seguimos sin saber si la regla del 20% de ampliación (pregunta 12 del levantamiento) aplica también a una falla adicional pequeña, o solo a una ampliación sobre trabajos ya autorizados. Mientras no se confirme, `ServicioDeAutorizacion` se modeló aplicando el umbral únicamente al segundo caso, dejando que toda falla adicional requiera autorización sin excepción.

5. Service Layer

A continuación, se presenta la tabla correspondiente en la cual se detallan los servicios de aplicación, sus métodos o casos de uso, parámetros de entrada, colaboradores, salidas y las reglas de negocio delegadas

Servicio	Caso de uso	Colaboradores	Salida	Reglas que delega
GestionDeIngresoService	CU-01 (previo)	IVehiculoRepository, IOrdenDeServicioRepository	OrdenDeServicio en Recibido	—
DiagnosticoService	CU-01 (previo)	IOrdenDeServicioRepository	Orden en Diagnosticado	Verificación en OrdenDeServicio
GenerarCotizacionService	CU-01	IOrdenDeServicioRepository	Cotizacion Original	No cotizar sin diagnóstico
AutorizarTrabajosService	CU-01, CU-02, CU-05	IOrdenDeServicioRepository, IClienteRepository	RegistroDeAutorizacion; trabajos Autorizado/Rechazado	validarAutorizador() en Cliente/ClienteFlotilla; umbral 20% en IPoliticaDeAutorizacion
ReportarFallaAdicionalService	CU-02	IOrdenDeServicioRepository	Cotizacion Complementaria	Reutiliza AutorizarTrabajosService
SolicitarRefaccionAProveedorService	CU-03	IOrdenDeCompraRepository	OrdenDeCompra	Solo si no hay existencia
MarcarEsperaDeRefaccionService	CU-03	IOrdenDeServicioRepository	Orden en espera de refacción	Reacciona a RefaccionSinExistenciaDetectada
NotificarNuevaFechaEntregaService	CU-03	IOrdenDeServicioRepository, INotificacionCliente	Confirmación de envío	—

CerrarOrdenDeServicioService	CU-04	IOrdenDeServicioRepository	Orden en Cerrado	Solo cobrar lo autorizado/ejecutado
GenerarFacturacionConsolidadaService	CU-05	IClienteRepository, IOrdenDeServicioRepository, IFacturacionConsolidadaRepository	FacturacionConsolidada	Agrupar solo Cerradas sin duplicados

5.1 Revisión de service Layer

¿El nombre del método expresa una acción del negocio?

R: Sí, cada método se define con verbos del negocio que representan una intención completa del usuario o de otro sistema (por ejemplo, confirmarPedido o aprobarCredito en lugar de nombres técnicos como updateDatos o save).

¿El método coordina o contiene decisiones que deberían estar en el dominio?

R: El método únicamente coordina el flujo: recibe la solicitud, obtiene los objetos necesarios, invoca el comportamiento del dominio, solicita la persistencia y devuelve un resultado. Todas las condiciones que describen lo que el negocio permite, calcula o protege se delegan a las entidades del dominio para evitar que la Service Layer acumule reglas.

¿El servicio depende de interfaces o de detalles concretos de base de datos?

R: Depende exclusivamente de interfaces (abstracciones de repositorios y servicios externos definidas en los colaboradores), manteniéndose desacoplado de motores de base de datos, consultas SQL o detalles de infraestructura.

¿La operación tiene un inicio y un final transaccional comprensible?

R: Sí, cada caso de uso opera como una unidad de trabajo clara que inicia al recibir los datos de entrada, ejecuta la lógica del dominio y concluye al asegurar la persistencia y devolver la salida (o revertir toda la operación si ocurre algún fallo).

¿Los errores del dominio pueden explicarse con términos del negocio?

R: Sí, cualquier validación o restricción incumplida devuelve excepciones o mensajes comprensibles en el lenguaje del negocio (por ejemplo, "Inventario insuficiente" o "Cuenta inactiva"), ocultando errores técnicos o de base de datos.

6. Repository

A continuación, se presenta la tabla de los Repositories, en la cual se definen las interfaces en términos del dominio, detallando sus necesidades, métodos propuestos y los objetos que recuperan o guardan sin exponer consultas SQL ni detalles de infraestructura.

Repository	Necesidad del dominio	Métodos propuestos	Objeto
IVehiculoRepository	Historial del vehículo	ObtenerPorId, ObtenerPorPlaca, Guardar	Vehiculo
IClienteRepository	Distinguir particular/flotilla, validar responsable	ObtenerPorId, ObtenerResponsableDeCuenta, Guardar	Cliente / ClienteFlotilla
IOrdenDeServicioRepository	Aggregate completo del servicio	ObtenerPorId, ObtenerPorVehiculo, Guardar	OrdenDeServicio
IOrdenDeCompraRepository	Solicitud a proveedor	ObtenerPorId, Guardar	OrdenDeCompra
IFacturacionConsolidadaRepository	Agrupar órdenes cerradas	ObtenerPorClienteYPeriodo, Guardar	FacturacionConsolidada

7. Unit of Work

En la siguiente tabla se describe la propuesta de Unit of Work, identificando las operaciones que deben completarse juntas, dónde inicia la transacción, qué cambios incluye y cómo se revierte la operación ante un error.

Operación	Qué incluye	Ante un error
AutorizarTrabajos (CU-01,02,05)	Varios Trabajo + RegistroDeAutorizacion, un solo Aggregate	Reversión completa; ningún trabajo queda en estado intermedio
SolicitarRefaccion (CU-03)	Solo OrdenDeCompra	No se crea la orden; reintentable sin afectar OrdenDeServicio
MarcarEnEspera (CU-03)	Solo OrdenDeServicio (estado)	El vehículo no cambia de estado; reintentable independientemente
GenerarFacturacion (CU-05)	Solo FacturacionConsolidada (referencias)	No deja ninguna orden inconsistente; reintentable

8. CQRS

A continuación, se presenta la tabla correspondiente en la cual se aplica CQRS básico para distinguir las operaciones que modifican el estado del sistema (*Commands*) de aquellas destinadas a la consulta de información (*Queries*), detallando sus datos de entrada, *Handlers*, resultados y justificación.

Tipo	Nombre	Handler	Resultado	Justificación
Command	AutorizarTrabajosCommand	AutorizarTrabajosHandler	Confirmación + RegistroDeAutorizacion	Cambia estado; crítico para BN-01
Command	ReportarFallaAdicionalCommand	ReportarFallaAdicionalHandler	Confirmación + Cotización complementaria	Cambia estado; dispara FallaAdicionalReportada
Command	CerrarOrdenDeServicioCommand	CerrarOrdenDeServicioHandler	Confirmación; orden Cerrado	Cambia estado final; dispara OrdenDeServicioCerrada

Query	ConsultarEstadoVehiculoQuery	ConsultarEstadoVehiculoHandler	EstadoVehiculoViewModel	US-14: lectura pura
Query	ObtenerDetalleParaCobroQuery	ObtenerDetalleParaCobroHandler	DetalleCobroViewModel	US-19: Caja lee solo lo autorizado/ejecutado
Query	ObtenerIndicadoresQuery	ObtenerIndicadoresHandler	IndicadoresViewModel	US-15: candidato a Table Module ligero

CU-05 (Facturación consolidada): No requiere separación adicional en Command/Query, ya que GenerarFacturacionConsolidadaService actúa como un *Command* de facto al ser un proceso de corte mensual de bajo volumen sin consultas de lectura competidoras.

RegistrarIngreso y RegistrarDiagnostico: Tampoco se modelaron como *Commands* explícitos adicionales debido a que son operaciones transaccionales directas ya coordinadas por la *Service Layer*, cuya complejidad y simetría entre escritura y lectura no justifican la sobrecarga de crear *Handlers* separados.

9. Revisión SOLID

En la siguiente tabla se muestra la evaluación arquitectural bajo los principios SOLID, identificando decisiones de diseño concretas para evitar responsabilidades mezcladas o acoplamientos indebidos, junto con la evidencia, el impacto y los ajustes aplicados antes de cerrar los diagramas.

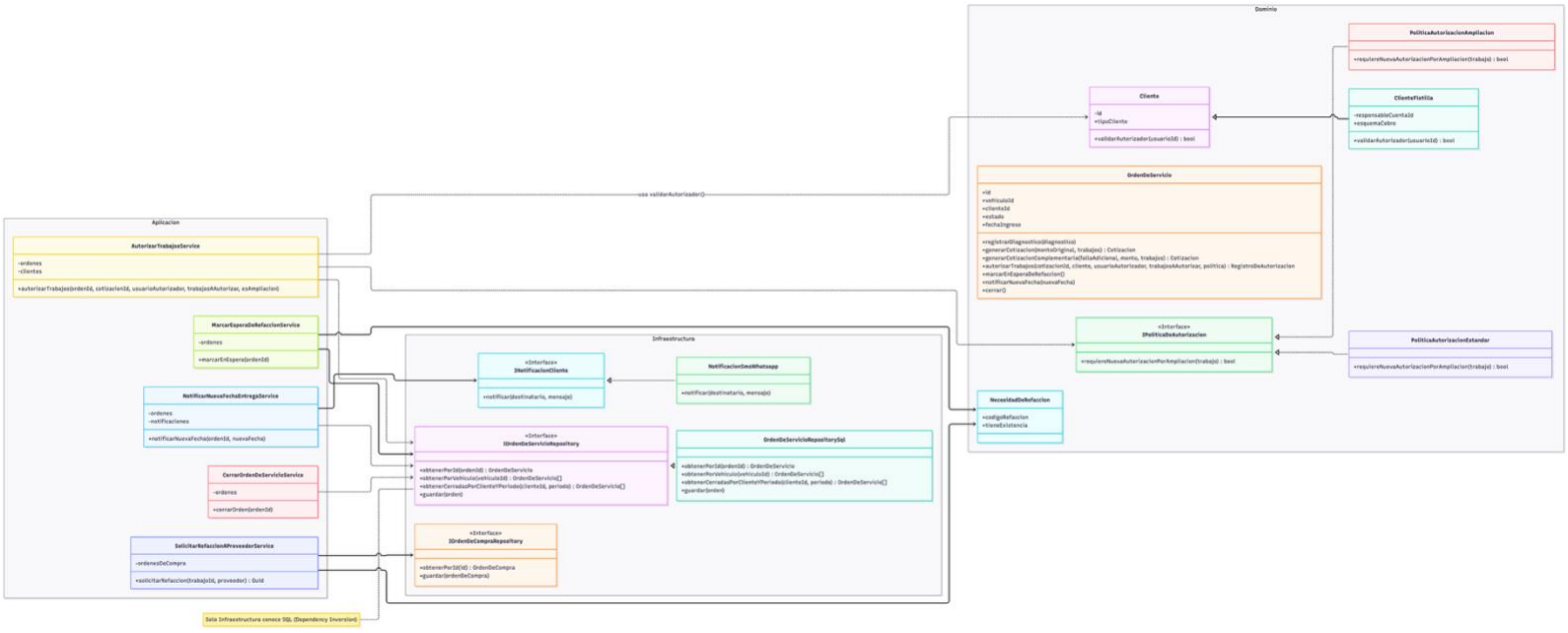
#	Principio	Hallazgo	Cambio realizado
1	*S*	Confirma buena decisión: AutorizarTrabajosService delega el umbral del 20% a ServicioDeAutorizacion	Ninguno
2	*S*	Corrección: un solo servicio coordinaba OrdenDeCompra y OrdenDeServicio (2 Aggregates)	Dividido en SolicitarRefaccionAProveedorService y MarcarEsperaDeRefaccionService
3	*O*	Riesgo: el umbral del 20% podía resolverse con condicionales crecientes	IPoliticaDeAutorizacion con políticas intercambiables por tipo

4	*L*	Corrección: servicio separado para flotillas duplicaba lógica en vez de sustituir	Eliminado; AutorizarTrabajosService reutilizado vía validarAutorizador() polimórfico
5	*I*	Riesgo: interfaz de notificación única con métodos por canal (SMS/correo/WhatsApp)	Interfaz mínima INotificacionCliente con un solo método genérico
6	*D*	Confirma buena decisión: todos los servicios dependen de interfaces de Repository	Ninguno

10. Representación de la arquitectura

10.1 Diagrama de clases

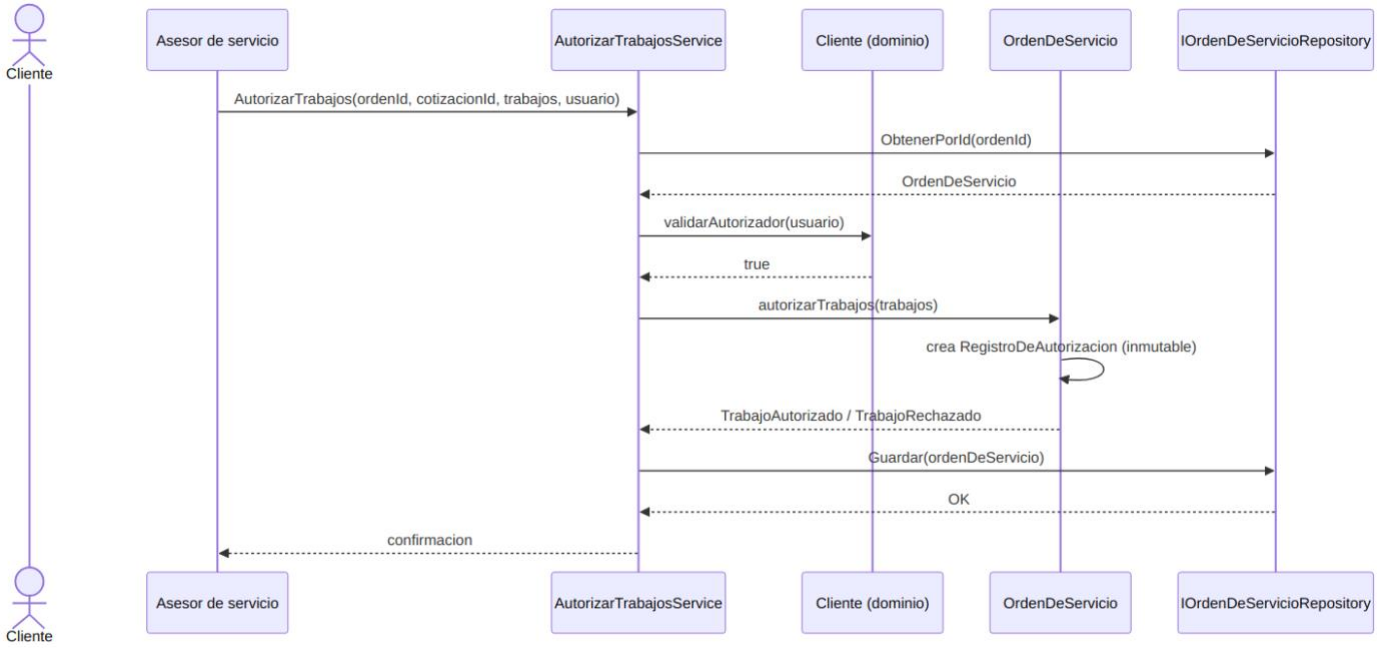
En el siguiente diagrama de clases se muestra la separación entre las capas de Aplicación, Dominio e Infraestructura de AutoFix Pro, ilustrando cómo los servicios de aplicación se comunican con las entidades del dominio y consumen las interfaces IOrdenDeServicioRepository e INotificacionCliente.



10.2 Diagramas de secuencia

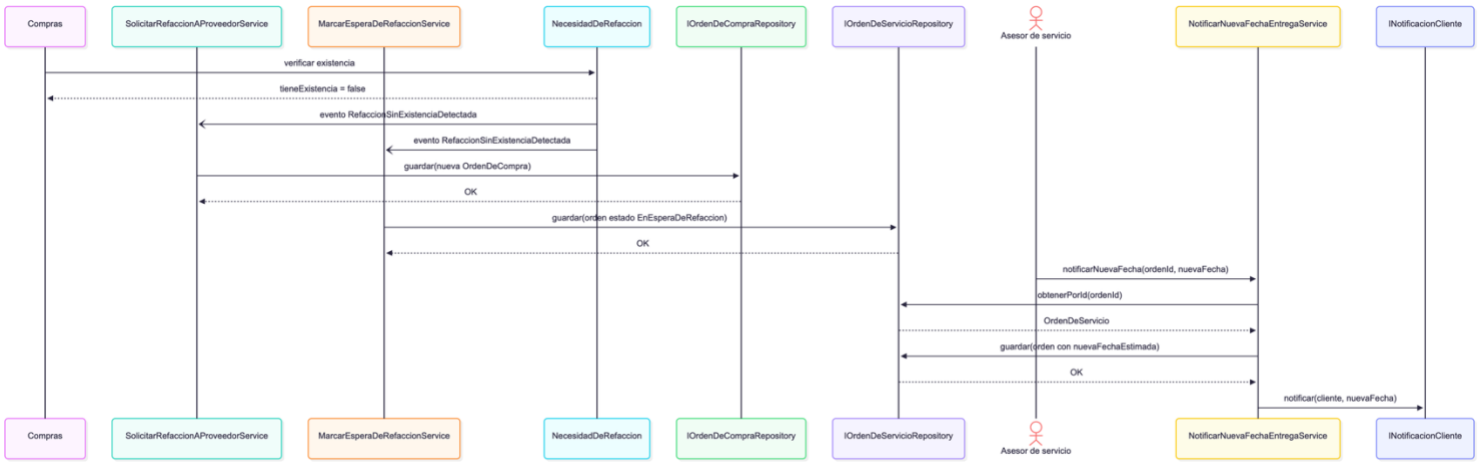
Flujo Normal

A continuación, se presenta el diagrama de secuencia del flujo normal (`CU-01`), el cual modela paso a paso la autorización de trabajos a través de `AutorizarTrabajosService`, la validación polimórfica en `Cliente`, la creación inmutable de `RegistroDeAutorizacion` en `OrdenDeServicio` y su persistencia.



Excepción

En el siguiente **diagrama de secuencia de excepción (`CU-03`)** se representa el flujo cuando `NecesidadDeRefaccion` detecta falta de existencia y emite el evento



`RefaccionSinExistenciaDetectada`, disparando de forma desacoplada la creación de la orden de compra, el cambio a espera de refacción y la notificación al cliente.

10.3 Architecture Decision Records

En la siguiente tabla se resumen los cuatro Architecture Decision Records (ADR) de AutoFix Pro, donde se justifican la elección de *Domain Model*, la división de servicios por *Aggregate*, la autorización polimórfica y las políticas intercambiables, junto con sus consecuencias y trazabilidad hacia la Unidad 2.

ADR	Título	Decisión	Consecuencias	Trazabilidad
001	Selección del patrón de lógica de negocio	Domain Model para el flujo; Table Module ligero para reportes	Mayor inversión inicial; reglas protegidas y extensibles	BN-06, US-03/04/08, pregunta 12
002	Separar gestión de refacción sin existencia en dos servicios	Dos servicios + Domain Event, en vez de transacción distribuida	Independencia entre Aggregates; consistencia eventual	BN-03, US-11/12/13, pregunta 9
003	Autorización polimórfica en Cliente/ClienteFlotilla	Método validarAutorizador() polimórfico, en vez de servicio separado	Un solo punto de entrada para autorizar; más mantenible	BN-05, US-16, pregunta 8
004	Política intercambiable para el umbral de autorización	Interfaz IPoliticaDeAutorizacion, en vez de condicionales crecientes	Regla nueva = clase nueva, no modificación	Pregunta 12; hallazgo Etapa 3

11. Validación de la propuesta

A continuación se presenta la matriz de validación final de la arquitectura de AutoFix Pro, en la cual se comprueba el cumplimiento de trazabilidad, lenguaje ubicuo, ubicación de reglas en el dominio, consistencia entre diagramas, desacoplamiento de persistencia, límites transaccionales y revisión de IA.

Validación	Pregunta	Resultado
Trazabilidad	¿Cada componente responde a una necesidad, regla o caso de uso?	Sí, salvo la política de "vehículo no recogido" (hallazgo pendiente), fuera de alcance hasta validarse
Lenguaje	¿Los mismos términos significan lo mismo en BRD, UML, código y ADR?	Sí; "Autorización Digital" se formalizó como RegistroDeAutorizacion
Responsabilidad	¿Las reglas están en el dominio con justificación?	Sí; el umbral del 20% vive en IPoliticaDeAutorizacion, no en la aplicación
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	Sí; ambos diagramas de secuencia usan los mismos servicios y repositorios del diagrama de clases
Persistencia	¿El dominio evita depender de SQL/ORM directamente?	Sí; todas las dependencias son interfaces de Repository
Transacción	¿El límite de la operación y su manejo de error están definidos?	Sí, con distinción ACID (autorizar trabajos) vs. eventual (refacción sin existencia)
IA	¿Cada aporte de IA fue revisado y su uso está registrado?	Sí; ver Bitácora de IA

12. Matriz de trazabilidad

Esta matriz conecta los problemas originales del negocio (BN) con las Historias de Usuario (US) que los resuelven, asegurando que no haya código huérfano (desarrollo sin propósito de negocio) ni necesidades sin atender.

ID Necesidad (BN)	Descripción de la Necesidad	Historias de Usuario (US) asociadas	Criterios de Éxito Impactados
BN-01	Reducir disputas por trabajos autorizados mediante evidencia digital.	US-02 (Fotos ingreso) US-07 (Registro digital de autorización) US-20 (Fotos entrega)	• Reducción medible de disputas en 3 meses. • 100% de autorizaciones con evidencia digital.
BN-02	Aumentar visibilidad del estado del vehículo en tiempo real.	US-14 (Consulta de estado) US-15 (Reportes gerente)	• Mejora medible de satisfacción en encuestas.
BN-03	Comunicar oportunamente retrasos por falta de refacciones.	US-11 (Orden compra) US-12 (Estado espera refacción) US-13 (Notificar nueva fecha)	• Tiempo promedio de 2 horas de comunicación de retrasos.
BN-04	Registrar fallas adicionales y su autorización.	US-09 (Reporte falla adicional)	• 100% de autorizaciones con evidencia digital.

		US-10 (Cotización complementaria)	
BN-05	Gestión de flotillas (autorización y facturación diferenciada).	US-16 (Autorización delegada) US-17 (Facturación mensual) US-18 (Prioridad contractual)	• Mejora medible de satisfacción del cliente (Flotillas).
BN-06	Estandarizar proceso de cotización total o parcial.	US-01, US-03, US-04(Flujo inicial) US-05, US-06(Aprobación parcial) US-08 (Visibilidad técnico)	• Reducción medible de disputas en 3 meses.

13. Bitácora de IA

Iteración	Objetivo	Prompt o enlace	Respuesta útil	Error o supuesto	Decisión del equipo
1	Analizar continuidad con la Unidad 2 antes de diseñar	Comencemos con la etapa número 1: ¿qué hallazgos encuentras en el documento de la Unidad 2 que sean importantes, hay correcciones que se deban realizar?	La IA identificó 5 hallazgos: carril del Gerente del taller ausente en el BPMN; brecha de política para vehículo no recogido (pregunta 14) sin US asociada; restricción de conexión inestable sin decisión de persistencia; contradicción entre pregunta 10 y pregunta 12 del levantamiento, mezcladas en US-10; y Autorización sin tipo de elemento de dominio definido.	La contradicción de US-10 (falla adicional vs. umbral del 20%) no estaba resuelta por el equipo en la Unidad 2; se registró como ambigüedad real, no se inventó una regla nueva para resolverla.	El equipo confirmó los 5 hallazgos y decidió avanzar con ellos declarados como supuestos pendientes.
2	Seleccionar y verificar los casos de uso de la Etapa 1	Por favor mantén la tabla con el formato ID, Caso de uso, Actor, Reglas relacionadas, Resultado esperado; verifica que los casos aparecen en las user stories o en el BPMN, al menos en uno de los dos	La IA propuso 5 casos de uso (CU-01 a CU-05) con su actor, regla y resultado, y hasta ese punto solo había comparado contra la captura de pantalla del BPMN, no contra el archivo fuente.	La IA reportó una discrepancia de actor entre el BPMN y las user stories (US-10 y US-17) basada en una lectura visual de la captura, que resultó ser un error de interpretación de la IA, no un error real del documento.	El equipo aportó los archivos fuente .drawio del BPMN y del UML para que la IA verificara contra el XML real en vez de la imagen; al revisar el archivo correcto, la IA retiró el hallazgo de discrepancia. El equipo también decidió agregar CU-05 (BN-05, sin cobertura en

					la propuesta inicial de 4 casos) y descartó un BPMN y un UML alternos (archivos .drawio adicionales) que no correspondían a la versión final de la Unidad 2.
3	Comparar patrones de lógica de negocio (Etapa 2)	Continuemos con la Etapa 2 donde vas a proporcionarnos la tabla donde se elege el patrón de la lógica de negocio, el formato debe mantenerse tal cual al documento y vas a agregar evidencia de la U2	La IA entregó la tabla comparativa (Transaction Script, Table Module, Domain Model) con 5 criterios y evidencia de la Unidad 2 en cada fila, y propuso Domain Model como patrón principal con Table Module ligero para reportes (US-15).	La actividad exige una decisión argumentada de 1 a 2 páginas; en la primera entrega solo se incluyó una oración de decisión, sin la justificación extendida.	El equipo aprobó la decisión (Domain Model) sin pedir cambios al razonamiento, y en una revisión posterior se agregó la justificación extendida de una página antes de la entrega final.
4	Construir el mapa de dominio (Etapa 3)	Ahora vamos a la etapa tres, dame un bosquejo del diagrama de dominio, respetando el los archivos que ya tienes, el diagrama debe de entregarse en codigo para mermaid, asi como tambien los demas requisitos que pide esta etapa (Entities, value objects, domain services, etc)	La IA entregó el diagrama Mermaid con Aggregates, Entities, Value Objects, Domain Services y Domain Events; el glosario actualizado; la ficha de cada Aggregate (raíz, responsabilidades, invariantes, eventos); y 6 decisiones dudosas o descartadas, incluyendo el umbral del 20% sin confirmar con el stakeholder.	Antes de este mapa, el equipo erroneamente subió un UML alterno (diagrama_uml_ed.drawio) que la IA detectó como inconsistente: le faltaban Trabajo, Autorización, ClienteFlotilla y FacturaciónConsolidada, y no distinguía tipo de cliente.	El equipo revisó la comparación que hizo la IA entre ambos UML y decidió descartar el archivo alterno, confirmando que el mapa de dominio construido con el UML del PDF original de la

					unidad 2 era el correcto.
5	Diseñar la Service Layer (Etapa 4)	Continuemos con la etapa 4, donde vas a diseñar la service layer respetando el formato de la tabla, debes de verificar obligatoriamente que cumpla con las 5 preguntas de revision para asegurar el cumplimiento de los requisitos de esta etapa	La IA entregó 9 Application Services con entrada, colaboradores, salida y reglas que delegan, y respondió explícitamente las 5 preguntas de revisión obligatoria (nombre de negocio, coordinación vs. decisión, dependencia de interfaces, límite transaccional, errores en términos de negocio).	—	El equipo aprobó la tabla sin cambios en este punto; sin embargo, las correcciones a esta etapa llegaron después, en la iteración 7.
6	Separar persistencia y transacciones (Etapa 5)	En esta etapa, nos proporcionaras las de interfaces de repository en terminos de dominio sin adentrarte en lenguaje SQL, Tambien, realizaras el bosquejo de la tabla de unit of work respetando el formato que solicita la actividad	La IA entregó las interfaces de Repository en términos del dominio (sin SQL) y la tabla de Unit of Work explicando dónde inicia cada transacción, qué incluye y qué pasa ante un error.	Quedó sin resolver si el Unit of Work debe contemplar la conexión inestable de las tablets mencionada en el BRD; se declaró como hallazgo pendiente y no se inventó una estrategia de reintento.	El equipo aceptó la propuesta y decidió dejar pendiente de validar la estrategia de conexión inestable.

7	Separar Commands y Queries (Etapa 6)	En esta etapa nos apoyaras proponiendo 3 commands y 3 queries separados, deben de contentes handler, entrada, resultado y justificacion basandote en el trabajo que ya existe de etapas anteriores, deberas cumplir los demas requisitos que solicita la actividad y justificar por que algunos CU no requieren esa separacion	La IA propuso 3 Commands y 3 Queries con handler, datos de entrada, resultado y justificación, con DTOs de lectura independientes del modelo de escritura, y justificó por qué CU-05 y el registro de ingreso no necesitan esta separación.	—	El equipo aprobó la propuesta sin cambios en este punto.
8	Revisar la propuesta con SOLID y aplicar correcciones (Etapa 7)	Sí (continuar a la Etapa 7) / después: ¿las correcciones dónde se deben hacer? / rediseña Etapa 3, 4 y 5 con las correcciones	La IA entregó 6 hallazgos (2 confirmaciones: S y D; 2 correcciones: S y L; 2 riesgos: O e I) con evidencia e impacto, y luego mapeó cada corrección a los documentos exactos que debían modificarse.	Las correcciones de SRP (dividir el servicio de refacciones), OCP (política de autorización intercambiable) y LSP (autorización polimórfica de flotillas) no estaban aplicadas en la primera versión de las Etapas 3 a 5.	El equipo pregunto a la IA explícitamente dónde debía aplicarse cada corrección antes de aceptarlas, y después solicitó el rediseño de las Etapas 3, 4 y 5 con las 3 correcciones ya incorporadas.
9	Representar la arquitectura: diagramas y ADR (Etapa 8)	entregame el bosquejo en codigo de mermaid para el diagrama de clases separado en capas mencionadas en el documento, tambien el codigo para un diagrama de secuencia en flujo normal y otro	La IA entregó el diagrama de clases por capas, dos diagramas de secuencia (flujo normal y excepción) y 4 ADR con los 7 campos exigidos (Título, Estado, Contexto, Alternativas, Decisión, Consecuencias, Trazabilidad).	El diagrama de clases inicial no incluía IOrdenDeCompraRepository ni NecesidadDeRefaccion, pese a que el diagrama de secuencia de excepción (CU-03) sí los usa como participantes, lo que violaba la consistencia exigida entre ambos diagramas.	El equipo pidió verificar el cumplimiento contra la rúbrica real de la actividad; tras detectarse la inconsistencia, se corrigió el

		para un excepcion, asi como tambien 4 ADR			diagrama de clases para incluir ambos elementos.
10	Validar la propuesta y preparar los entregables formales (Etapa 9 y entregables)	Valida los requisitos del documento de la U3 vs la informacion que me has entregado, si todo esta correcto entregame los documentos para yo poderlos editar manualmente y un codigo sencillo base en algun lenguaje de programacion facil de desplegar	La IA entregó la validación de las 7 preguntas de la Etapa 9 y generó el Informe de Arquitectura (Word), la Matriz de Trazabilidad (Excel) y un código base en C#, con los diagramas renderizados en PNG.	-	El equipo solicitó verificar cada etapa contra la rúbrica real de la actividad, corregir lo que no cumpliera, crear esta bitácora con el formato correcto para rellenar, y rehacer el código base en React con JavaScript en su versión más reciente en lugar de C#, ya que se decidio desplegar el proyecto como web para facilitar su uso

14. Conclusiones

El proceso de diseño partió de un análisis crítico de la Unidad 2, lo que permitió detectar y corregir una ambigüedad real entre el levantamiento y las user stories, y descartar material fuente que no correspondía a la versión final entregada. La aplicación de Domain Model, combinada con una revisión explícita contra los principios SOLID (Etapa 7), llevó

a dos correcciones concretas de diseño, separar la gestión de refacciones en dos servicios de un solo Aggregate, y unificar la autorización de clientes particulares y de flotilla mediante polimorfismo; documentadas formalmente como ADR. El código base adjunto materializa estas decisiones en REACT de js, incluyendo comentarios que referencian explícitamente cada ADR y cada corrección de la Etapa 7.